

* ACID Properties :-

- ↳ It is a set of 4 rules that makes transaction reliable, consistent and crash-proof.
- ↳ Transaction is a group of db ops that are treated as single unit or work. In either Ending succeeds or fails.

① Atomicity (All or Nothing) :-

- ↳ Either the whole txn happens or nothing happens.
- ↳ No partial updates. (Commit or Rollback).

② Consistency (Valid Data) :-

- ↳ It ensures that every txn brings the db from one valid state to another valid state by enforcing constraints & business rules.
- Eg:- If balance = 100 and sending 500 → Rollback from constraints.

③ Isolation (No interference) :-

- ↳ Multiple txn should not interfere with each other.
- ↳ But it arises issues like Race Condition (Read lock), Dirty read, etc.

④ Durability (Permanent Save) :-

- ↳ Once txn is complete → data is saved permanently.

* Isolation Levels :-

- ↳ It decides how much one txn can see/interfere with another txn.

- Issues :-

① Dirty Read (un-committed data)

- ↳ Reading data that has been changed but not yet committed by another txn.

Start at same time
T1 updates balance → 4000 (not committed)
T2 reads → 4000
T1 rolls back → balance goes back to 5000
T2 saw invalid data

(ii) Non-Repeatable read (Same row changes)

- ↳ Reading the same row twice and getting diff values bcz another txn updated in between. (I=hi)

(iii) Phantom Read (new rows appear/disappear)

- ↳ When query returns diff rows bcz another txn inserted or deleted the data.

- UX :-

① Banking / Payments → Serializable / Repeatable read

② E-commerce → Repeatable read

iii) Social Media $\rightarrow$ Read Committed.

— Level

Level	Mixy Read	Non-Repeatable	Phantom	Speed
i) Read Uncommitted	X	Possible X	X	Fastest
ii) Read Committed (pg default)	Prevented $\checkmark$	X	X	Fast
iii) Repeatable Read (mysql II)	$\checkmark$	$\checkmark$	X	Medium
iv) Serializable	$\checkmark$	$\checkmark$	$\checkmark$	Slowest

$\rightarrow$ T1 runs completely then only T2 runs $\rightarrow$ no overlap, no issues.

— SET TRANSACTION ISOLATION LEVEL REPEATABLE READ.

* Locking: —

$\rightarrow$ Preventing multiple txn from modifying the same data at same time.

— Types: —

i) Shared Lock (Read Lock)

$\rightarrow$ Multiple txn read the same data at same time but no one can modify it.

ii) Exclusive Lock (Write Lock)

$\rightarrow$ Only one txn to read & write, and block all others.

$\rightarrow$ No one can access (Read/Write) until T1 finishes.

— Strategy of Handling Race Conditions: —

i) Pessimistic Locking (Row level lock): —

$\rightarrow$ Assume conflict will happen $\rightarrow$ lock the data before updating.

Select * from table for update

ii) Optimistic Locking

$\rightarrow$ Assume conflict is rare $\rightarrow$ don't lock, just check version before updating.

$\rightarrow$ Retry logic needed for T2 txn.

* Deadlocks: —

$\rightarrow$ It occurs when txn wait on each other in a circular dependency.

txn A: Lock Row (id=1)
Wait for Row (id=2)

txn B: (id=2)
(id=3)
Deadlock

$\rightarrow$ Database like Pgsql detect deadlock automatically, kill one txn and throw error: Deadlock detected.

$\rightarrow$ It can be prevented by maintaining same lock order, keep txn short or using FOR UPDATE NOWAIT.

* B-tree:-

- ↳ A balanced tree data structure used to store indexes efficiently.
- ↳ by default in mysql & postgresql.
- ↳ Index allows the db to find the rows quickly without scanning the entire table - like the index at the back of book.

* Query optimization:-

- ① Use EXPLAIN ANALYZE select * from _____ → to check how your query is executed.

↳ Seq Scan on users → Full table scan. $O(n)$.

↳ Index Scan using 'idx_email' → Index scan. $O(\log n)$.

- ② Avoid Index ~~or~~ Killer - function on column.

↳ where YEAR(created_at) = 2024 → Full Scan

↳ where created_at between '2024-01-01' and '2024-12-31' → Index Scan.

- ③ Log Slow queries:-

SET log_min_duration_statement = 1000 (1 sec).

- ④ Use JOIN instead of sub-query

- ⑤ Use single bulk insert instead of 1000 individual inserts.

* N+1 Problem:-

- ↳ 1 query to fetch list + N queries for each item for loop.

Total = N+1 queries.

- Solutions:-

- ① Use JOIN → 1 query.

- ② 2-Query batching

* Pagination Strategy:-

- ① OFFSET / LIMIT - simple but slow

↳ select * from posts order by id LIMIT 20, OFFSET 0 → Page 1.
" " " 20 " 20 → Page 2.

↳ On 10 million rows → very slow.

- ② cursor based pagination - very fast (used in Instagram, Twitter)

↳ Instead of skipping rows, use the last seen id (cursor).

↳ select * from posts order by id desc limit 20;

↳ return id from 1000, _____ to 981.

select * from posts where id < 981 → last seen id
order by id desc limit 20; } returns 980, 979, _____ 961.



* CTEs (Common Table Expressions):-

- ↳ It improves query readability by breaking complex queries into steps.
- ↳ CTE = a temp table inside a query

eg:- With high-salary-users AS (

SELECT * FROM users where salary > 5000 } filter users

SELECT * FROM high-salary-users where age > 25 } use filtered data

* Window Function:-

↳ It allows calculations across rows without collapsing them. like group by

- (1) ROW_NUMBER() → given unique numbers to each row.
- (2) RANK() → Same rank for same values, but skips next numbers.
- (3) LAG() → Gets previous row value
- (4) DENSE_RANK → Same rank for same values, but no skip.
- (5) LEAD() → Gets next row value
- (6) PARTITION BY → divide rows into groups, allowing calculations within each group without collapsing the result set

eg:- SELECT name, salary, dept

ROW_NUMBER() OVER (ORDER BY salary desc) as row_number

RANK()

as rank

DENSE_RANK()

as

rank-dense

LAG(salary)

OVER (ORDER BY salary)

as Prev-salary

RANK() OVER (PARTITION BY dept ORDER BY salary desc) as rank

FROM users;

* Caching Strategies:-

- (1) Cache-Aside → check cache if not found, get from db and put it in cache
- (2) Write-Through → update db and cache together
- (3) Write-Behind → update cache immediately and update db in queue
- (4) TTL → use proper expiry time
- (5) Redis Data Structure:-
 - redis.set() → simple cache
 - redis.hset() → store object
 - redis.lpush() → list

redis. sadd() → Set unique values
redis. zadd() → Sorted Set

* CAP Theorem:—

Let's state that in a distributed system, we can only guarantee 2 out of 3 properties. Since network partitions are unavoidable, we must choose between CP and AP.

— The 3 Properties:—

- (i) Consistency (C) → Every read gives you latest data
- (ii) Availability (A) → Every req gives response not error (503). Response data can be old (stale).
- (iii) Partition Tolerance (P) → System should work even with network Partition (Network failure b/w servers). This is unavoidable.

— Real Rule:—

(i) CP System

↳ Always correct data but may reject requests.
Eg → Critical APIs, banking systems, payments, inventory, etc

(ii) AP System

↳ Always respond, but data may be slightly outdated.
Eg → Social media feeds, analytics dashboard, Cart, etc

* Connection Pooling:—

↳ It improves performance by reusing db connection instead of creating a new one for every req, reducing overhead

Without pooling

Req → open db connection → Query → close connection

expensive

} - slow response
- db crash risk

With pooling

Req → get connection from pool → Query → Return to pool.

} - Fast
- Efficient

* DB Migration:—

↳ Version controlled scripts that describe incremental changes to your db schema over time. with roll back support

— Steps:—

(i) create a migration script file.

or sqlize migration: generate --name (name of file).

(ii) write script in up migration for apply changes
write script in down migration for rollback

- (iii) Run Migration → `npm sequelize-cli db:migrate`
 (iv) Undo Migration → `npm sequelize-cli db:migrate:undo`

* Stored Procedure: - (DB ke andar mini backend logic)

- ↳ It is a named, pre-compiled set of SQL statements stored in the database that can be called by name.
- ↳ It provides reusability of same query, performance and security.
- ↳ Keep business logic in backend.
- Use stored procedure only for heavy db operations.

* MVCC (Multi-Version Concurrency Control): -

- ↳ It allows multiple version of a row to exist so that read ops don't block write ops, high concurrency.
- ↳ User may get stale (old) data.

* Consistent Snapshot

Snapshot → Consistent Snapshot → Consistent Snapshot

* DB Migration

1. Create a migration file.
2. Run the migration.
3. Undo the migration.